1. System-design workflow

1. Clarify users, core actions, read/write ratio, freshness, ordering, privacy, availability, latency and retention.
2. Estimate scale: average/peak QPS, object size, storage growth, bandwidth, concurrency and hot keys.
3. Define APIs, entities, IDs, indexes, pagination, idempotency and error behavior.
4. Find the bottleneck: CPU, DB, network, disk, lock, queue or hot partition.
5. Choose primitives: cache, CDN, LB, queue, partitioning, replication, object store, search.
6. Close the loop: failure modes, recovery, metrics, logs, traces and SLOs.

Takeaway: Every design answer should connect requirement → workload → bottleneck → mechanism → failure/performance behavior.

2. Sizing formulas

$$\text{QPS}_{avg} = \frac{\text{requests}}{\text{seconds}}; \qquad \text{QPS}_{peak} = p\,\text{QPS}_{avg}\ (p \approx 2\text{–}5)$$

$$\text{storage} = \text{objects} \times \text{bytes/object} \times (1 + \text{overhead}) \times \text{replicas}$$

$$\text{bandwidth} = \text{QPS} \times \text{payload}; \qquad \text{concurrency} \approx \text{QPS} \times \text{latency}$$

- 1 month $\approx 2.6 \times 10^6$s; 1 day = 86,400s; 1 billion $\approx 10^9$.
- Rough servers: $\lceil$peak QPS/(capacity per server × target utilization)$\rceil$.
- Replicas improve availability/read capacity but multiply storage and write coordination.

Worked: URL shortener

500M new URLs/month, read:write = 100 : 1. Writes ≈ 193 QPS; reads ≈ 19,300 QPS. Over 5 years: 30B URLs. At 500B/row, raw = 15TB; with 3× replication ≈ 45TB before indexes. Cache hot redirects; analytics async.

3. Architecture blocks

Client/CDN	browser cache, static content, geo-local delivery
LB/proxy	health checks, TLS, routing, rate limits, hiding backends
App tier	stateless horizontal replicas, validation, auth
Cache	low-latency/stale-tolerant reads; TTL/LRU; invalidation
DB	source of truth; indexes, partitions, transactions, replicas
Queue	async work, buffering, backpressure, retry/replay
Object store	large blobs; DB stores metadata/object ID
Search	derived text index; not the source of truth

4. Partitioning and replication

Range	scans/time windows; newest range can hotspot.
Hash	uniform point lookup; poor range scans.
Consistent hash	fewer keys move after membership changes; vnodes balance load.
Leader–follower	simple ordering; leader write bottleneck/failure target.
Active–active	scale writes/reads; conflicts and consistency harder.

Exam cue: Adding app servers does not fix a saturated database. Reduce DB work with indexes, cache, read replicas, partitioning or precomputation.

5. High-value design principles

- Modularity limits failure propagation; abstraction separates interface from implementation.
- Layering/hierarchy reduce interactions; naming/indirection enables replacement.
- Design for iteration: start small, measure, preserve conceptual integrity; avoid the second-system effect.
- Robustness: tolerate expected input variation; fail fast on invalid state.
- Read-heavy system: write to canonical store, build read models asynchronously, route reads to cache/replicas; state staleness.

6. Sample Q1: Citizen 360

Five DBs each contain $N = 10^9$ records. A scholarship needs identity, 12th pass date, current education/employment and family income.

Entity resolution

Prefer Aadhar ID as canonical key. If missing, use a deterministic match (name + DOB + address/phone) only as a candidate, then confidence/fallback/manual resolution. Store a canonical citizen ID and provenance.

Naive CHECK_ELIGIBILITY

Scan every table per request: $\Theta(5N)$ record examinations; joins/remote calls add network and serialization cost. Name matching is not a safe unique key.

Optimized CHECK_ELIGIBILITY

Indexes/hash maps keyed by Aadhar ID: $O(k \log N)$ with B-tree indexes or expected $O(k)$ with hash indexes, k = sources used. Better still, maintain a canonical/materialized citizen profile updated when yearly source data changes.

NOTIFY_SCHEME

Do not re-run five scans for every notification. Maintain an eligibility index/materialized set. One batch scan costs $O(N)$ once per update, or $O(K)$ to notify K eligible citizens; batch and make sends idempotent.

Exam cue: State missing/obsolete fields, yearly update assumption, canonical ID, indexes/materialized views, and notification retry/idempotency.

7. Sample Q2a: SQL/NoSQL T/F cues

i	NoSQL can use dynamic schemas.	**T**
ii	SQL is powerful for defining/manipulating data.	**T**
iii	NoSQL is less cost-effective for big data than SQL.	**F**
iv	NoSQL is usually best when financial integrity is essential.	**F**
v	SQL is best for rapidly changing application/data structures.	**F**
vi	Dynamo always writes to the first N preference-list nodes.	**F**
vii	LSM trees have lower append write amplification than B-trees.	**T**

8. Sample Q2b–c: application choices

TrueCaller spam flag

Use a key-value/inverted lookup keyed by phone number: maintain spam score/report count, recent reports, and rate-limit updates. A point lookup is $O(1)$ expected; do not scan all users/calls. For abuse resistance, weight trusted reports and deduplicate reporters.

Fintech firm-wide loan view

Prefer a document/relational system with strong consistency, transactions, secondary indexes and aggregation (sample choices: MongoDB over DynamoDB when the query needs integrated views and flexible document-level access). Dynamo is preferable for predictable key-value access at massive scale, not ad-hoc cross-entity views.

9. Sample Q2d–f: Dynamo answers

- **Zero-hop DHT:** clients know the partition map and contact the owning server directly.
- Temporary replica unavailable and update stored elsewhere: **hinted handoff**.
- Vnode benefits: spreads load, speeds rebalancing, supports heterogeneous machines, reduces hotspots. It does **not** mean virtual machines.
- Sloppy quorum = write to healthy substitutes; hinted handoff = later forward the hint to the intended replica.

10. Sample Q2g–j: C-Store + LSM

C-Store WOS/ROS

WOS (write-optimized store) accepts incoming tuples. Periodically move/merge tuples into ROS (read-optimized store), sort/compress and update projections. Queries use ROS for scans; WOS is also checked until tuples are moved. Movement bounds WOS growth and improves compressed reads.

Compression

Dictionary encoding maps repeated values to compact codes; RLE compresses runs; delta encoding stores differences. Query directly on compressed data when the predicate/operation can be evaluated on the encoded representation (e.g., dictionary equality or RLE range/run metadata).

WAL in an LSM tree

Append the update to a durable write-ahead log before acknowledging; memtable can be reconstructed after a crash, then flushed/compacted into sorted runs.

11. Storage and consistency quick facts

RAID-0	striping; capacity/performance, no failure tolerance.
RAID-1	mirroring; strong reads/reliability, half usable capacity.
RAID-4/5	parity; one-disk tolerance; small writes pay read-modify-write; RAID-5 distributes parity.
LSM	fast sequential writes; compaction/read/space amplification trade-offs.
WAL	durability + crash recovery before index/table update.
Quorum	if $R + W > N$ (under assumptions), read/write sets intersect.
Merkle tree	compare range hashes to locate divergent replicas.

12. Mini applications to reuse

Newsfeed

Fan-out-on-write for normal users; fan-out-on-read for celebrities. Partition by user ID, salt hot users, cursor pagination $(timestamp, ID)$, cache first page, media in object store, ranking async.

Chat

Conversation ID is partition key; append messages, assign monotonic per-conversation sequence, use WebSocket/gateway for delivery, durable queue for notifications, idempotent message IDs, offline replay from log.

File sync / drive

Metadata DB maps file/version to object-store blob; chunk large files, content hash for dedup, background replication, conflict/version policy, CDN for downloads, resumable uploads.

Exam cue: When the question changes, keep the same structure: requirements, scale, API, key, storage, bottleneck, failure path, observability.

1. Queueing notation

λ	arrival rate (jobs/s)
S	service time/requirement; $E[S]$ mean service time
μ	service rate $= 1/E[S]$
ρ	utilization; M/M/1: $\rho = \lambda/\mu$
L, L_q	mean jobs in system/queue
W, W_q	mean time in system/queue
X	throughput (completed jobs/s)
R	resource requirement $= \lambda E[S]$

$$\text{M/M/1 stable iff } \rho = \lambda E[S] < 1$$

2. Little's Law + M/M/1

$$L = \lambda W; \qquad L_q = \lambda W_q$$

$$W = \tfrac{1}{\mu-\lambda} = \tfrac{E[S]}{1-\rho}; \qquad W_q = \tfrac{\rho}{\mu-\lambda}$$

$$L = \tfrac{\rho}{1-\rho}; \qquad L_q = \tfrac{\rho^2}{1-\rho}; \qquad P_n = (1-\rho)\rho^n$$

- W = waiting + service. As $\rho \to 1$, delay explodes.
- Be precise: "2 customers in system" uses P_2; "2 waiting" uses P_3 in M/M/1.

Worked M/M/1

$\lambda = 8$/s, $\mu = 10$/s: $\rho = .8$, $W = 1/(10-8) = .5$s, $L = 8(.5) = 4$, $W_q = .5 - .1 = .4$s.

3. Multi-server / staffing

$$R = \tfrac{\lambda}{\mu} = \lambda E[S] \quad \text{(offered load)}$$

$$k > R \text{ for stability}; \qquad k \approx R + \beta\sqrt{R} \quad \text{(square-root staffing)}$$

$$B(k,R) = \frac{R^k/k!}{\sum_{i=0}^{k} R^i/i!} \quad \text{(Erlang-B, no waiting)}$$

$$C(k,R) = \tfrac{R^k}{k!}\tfrac{k}{k-R}P_0 \quad \text{(Erlang-C, probability an arrival waits)}$$

- For M/M/k with a queue: $P_{wait} = C(k,R)$ and mean queue wait $W_q = P_{wait}/(k\mu - \lambda)$.
- For M/M/k/k: no queue; blocked/lost probability is Erlang-B.

4. Variability / bus-stop formula

$$E[\text{residual wait}] = \tfrac{E[S^2]}{2E[S]} = \tfrac{E[S]}{2}(1 + C_S^2); \qquad C_S = \tfrac{\sigma_S}{E[S]}$$

- Inspection paradox: random observers see long inter-arrival intervals more often.
- High variance increases delay even if the mean arrival/service rate is unchanged.

Worked bus

$E[S] = 10$ min, $\sigma = 30$ min: $E[S^2] = 10^2 + 30^2 = 1000$; wait $= 1000/(2 \cdot 10) = 50$ min.

5. Operational laws / closed systems

$$U_i = X_i S_i = X D_i; \quad X_i = V_i X; \quad D_i = V_i S_i; \quad D = \sum_i D_i$$

$$X \le \tfrac{1}{D_{max}}; \qquad X = \tfrac{N}{R+Z}; \qquad R = \tfrac{N}{X} - Z$$

- V_i visits/job; S_i service/visit; D_i demand/job; Z think time; N closed-system users.
- Improve bottleneck demand D_{max} first at high N; balance loads where possible.

6. Sample Q3a–c: solved

3a. Tech Cafe

Empty 20% $\Rightarrow P_0 = 1 - \rho = .2 \Rightarrow \rho = .8$. Thus $P_2 = (1-\rho)\rho^2 = .2(.8)^2 = .128 = 12.8\%$.

3b. Bus

Use residual-life formula, not just half the mean: $E[S^2] = \sigma^2 + (E[S])^2 = 30^2 + 10^2 = 1000$ min^2. Wait $= E[S^2]/(2E[S]) = 50$ min.

3c. Google Colab

Arrival $\lambda = 10$/s; active work $E[S] = 30$ min $= 1800$s. Resource requirement:

$$R = \lambda E[S] = 10 \cdot 1800 = 18{,}000 \text{ vCPU}$$

For 90% immediate service using square-root staffing, $z_{.90} \approx 1.282$:

$$k \approx R + z\sqrt{R} = 18{,}000 + 1.282\sqrt{18{,}000} \approx 18{,}172 \text{ vCPU}$$

With 200 vCPU/server: $\lceil 18{,}172/200 \rceil = \boxed{91 \text{ servers}}$ (before extra failure/headroom policy).

Exam cue: If the instructor instead specifies a strict capacity model rather than square-root staffing, state that assumption. The key first step is $R = 18{,}000$.

7. Sample Q3d: Indigo M/M/4

Given $\lambda = 2$/min, $\mu = 1$/min/agent, $k = 4$. Offered load $R = \lambda/\mu = 2$, $\rho = R/k = .5$.

$$P_0 = \left[\sum_{n=0}^{k-1} \tfrac{R^n}{n!} + \tfrac{R^k}{k!}\tfrac{1}{1-\rho}\right]^{-1} = \tfrac{1}{8.6667} = .1154$$

$$P_{wait} = \tfrac{R^k}{k!}\tfrac{1}{1-\rho}P_0 = 1.3333(.1154) = .1538$$

i	Empty roughly 50%? Actual $P_0 = 11.54\%$.	**False**
ii	Kiosk probability > no-kiosk wait probability? Erlang-B $B(4,2) = .6667/7 = .0952 < .1538$.	**False**
iii	Reduce to $k = 3$: $\rho = 2/3$, $P_{wait} = .4444$, $W_q = .4444/(3-2) = .4444$ min vs current $.1538/(4-2) = .0769$ min.	**False**

Exam cue: Always justify T/F with the probability or mean, not only the label.

8. Dispatch / load balancing

FCFS	fair/simple; long jobs delay short jobs.
PS	time-share; good for preemptible requests.
SRPT	minimizes mean response when sizes known/preemptible; may starve long jobs.
JSQ	shortest queue; strong for preemptible/low-variance work.
LWL	least remaining work; good with variable sizes.
SITA	isolate short/medium/long jobs under high variance.
RR	good for homogeneous servers and regular jobs.
Power of two	sample two random queues, pick shorter; cheap and effective.
Central queue	reduces idle servers; dispatcher/queue must scale.

9. Faults, errors, failures

- Fault = latent defect; error = incorrect state/signal; failure = wrong behavior at the interface.
- Response chain: detect → contain → mask/recover → report/learn.
- Detection: checksums, timeouts, health probes, invariants, replica disagreement.
- Containment: modules, bulkheads, process/rack/region boundaries.
- Masking: retries, replicas, parity/erasure, failover, correction.
- Fail-fast reports invalid state; fail-stop halts rather than corrupting output; fail-soft serves partial functionality.

10. Availability + resilience formulas

$$A = \tfrac{\text{uptime}}{\text{uptime}+\text{downtime}}; \qquad A_{series} = \prod_i A_i; \qquad A_{parallel} = 1 - \prod_i (1 - A_i)$$

99%	14.4 min/day downtime
99.9%	1.44 min/day
99.99%	8.64 s/day
99.999%	0.864 s/day

- Parallel formula assumes independent failures; same rack/power/region/software bug breaks independence.
- Explicit fault model: identify risk, add detection/containment/masking, observe masked faults, perform postmortems.

11. Resilient request path

1. Deadline/timeout on every network hop.
2. Retry only transient errors and idempotent operations; exponential backoff + jitter; cap attempts.
3. Circuit breaker after repeated dependency failure; recover gradually.
4. Bulkhead resources by tenant/function; prevent one dependency from taking all threads.
5. Rate-limit/load-shed/load-level before queues become unbounded.
6. Deduplicate with request/message IDs; use durable queue for async retries.

12. Observability

Metrics	rate, errors, latency p50/p95/p99, saturation, queue depth
Logs	structured events with correlation ID; privacy-aware
Traces	request path across services; find fan-out/slow spans
SLI	measured signal, e.g. successful requests under 200ms
SLO	target for SLI; alert on actionable error-budget burn

13. One-minute exam answer template

1. State assumptions, units, workload and target metric.
2. Write formula/law before substituting numbers.
3. Check stability/utilization and identify bottleneck.
4. For design: API, data model, key/index, partition, replicas, cache, queue.
5. Explain consistency/freshness, failure path and observability.
6. End with trade-off and capacity/headroom.

Exam cue: Common traps: confusing service time with response time; using average instead of tail latency; forgetting queueing/replication overhead; naming a technology without explaining its access pattern.